

AMR MAP EDITOR

USER MANUAL

ROS2 • Fleet Manager • Open-RMF / Traffic Editor

Item	Details
Project guide base	v0.11.0
Manual scope	Import, edit, connect paths, configure RMF, validate, save, and export
Latest updates	Shared junction behavior, lane-vertex deduplication, .amrmap lane orientation persistence
Reference coordinates	Exactly 4 corner pairs; RMF values in meters
Updated	2026-09-17

What changed in this revision

The normal user workflow is unchanged. The editor/exporter now handles connected-lane junctions more robustly and prevents duplicate RMF graph vertices when connected endpoints are only a few millimeters or centimeters apart.

1. Application Overview

AMR Map Editor is a CAD/GIS-style web application for creating and maintaining AMR navigation maps. It supports ROS occupancy maps, navigation objects, paths, operational zones, robot configuration, and Open-RMF / Traffic Editor building geometry.

Stage	User action
Import	Load ROS map YAML + image or a supported project/map file
Edit Map	Clean occupancy data if required
Objects	Create waypoints and stations
Paths	Create navigation lanes, connect junctions, set travel direction and orientation
Zones	Create No-Go, Slow, Loading, Safety, etc.
Building	Create Floor, Wall, Door, Model, Measurement
Robot	Configure footprint, clearance, turning radius, max speed
Validate	Check topology and map constraints
Save / Export	Save .amrmap and export ROS / JSON / GeoJSON / RMF outputs

1.1 Quick Start

```
npm install
npm run dev
```

1.2 Recommended Workflow

1. Import map.
2. Verify resolution, origin X/Y, and origin yaw.
3. Clean the occupancy map if needed.
4. Create navigation objects.
5. Draw paths and connect lane junctions.
6. Set lane travel direction and Forward/Backward orientation.
7. Create zones.
8. Create RMF Floor and other building geometry if RMF export is required.
9. Configure robot.
10. Validate.
11. Save .amrmap and export the required deployment formats.

2. Importing a Map

For a ROS map, import the YAML and occupancy image together. After import, confirm the map metadata before building navigation or RMF geometry.

```
map.yaml
map.pgm
```

Metadata	Why it matters
Resolution	Meters per pixel; used by RMF and reference-coordinate conversion
Origin X / Y	Places the image in the ROS/world frame

Origin Yaw	Rotates image/world coordinate relationship
Width / Height	Defines map bounds and reference-image conversion

Important

Incorrect resolution or origin produces incorrect RMF geometry and reference coordinates.

3. Navigation Objects

- Waypoint
- Home
- Charging Station
- Docking Station
- Pickup
- Drop-off
- Waiting
- Parking

The Properties panel supports name, ID, type, X/Y in meters, heading/yaw, description, and enabled state.

3.1 Pickup / Drop-off RMF Properties

Object	Property	RMF field
Pickup	pickup_dispenser	pickup_dispenser: [1, value]
Drop-off	dropoff_ingestor	dropoff_ingestor: [1, value]

4. Paths and Lane Connectivity

Supported path types include Normal, Preferred, One-way, Bidirectional, and Restricted.

4.1 Travel Direction

Setting	Meaning	RMF
One-way	Travel A -> B only	bidirectional: false
Bidirectional	Travel A <-> B	bidirectional: true

4.2 Lane Orientation

Orientation	Meaning	Export
None	No heading constraint	orientation: [1, ""]
Forward	Robot faces along stored A -> B order	orientation: [1, forward]
Backward	Robot faces opposite stored A -> B order	orientation: [1, backward]

Travel direction and orientation are different

One-way/Two-way controls where the robot may travel. Forward/Backward controls how the robot should face while traversing the lane.

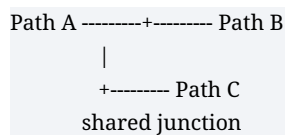
4.3 Connecting Paths

A visual crossing is not automatically a topology connection. Use endpoint snapping or the Connect controls so the paths share a navigation junction.

- Connect Start
- Connect End
- Connect Both
- Endpoint drag-and-snap
- Snap to an existing path vertex
- Snap to the middle of another path segment (junction insertion)

4.4 Shared Junction Behavior (Updated)

When two or more paths connect at the same location, the editor treats the location as one logical shared junction. The user should not create multiple manual points at the same physical location.



- Connected path endpoints are snapped to a canonical coordinate.
- Dragging a shared junction keeps connected path geometry attached to the same logical location.
- The UI should show one effective junction location rather than requiring the user to manage duplicate points.
- No extra user step is required to merge duplicate lane points before export.

User workflow unchanged

Continue drawing and connecting paths normally. Duplicate lane-vertex cleanup is handled by the editor/exporter logic automatically.

5. RMF Building Geometry

Tool	Use
Wall	Physical/RMF boundary
Door	RMF door definition
Floor Area	RMF floor polygon and reference-coordinate source
Model	Static/dispensable model
Measurement	Scale/distance metadata

5.1 Floor Area

For automatic reference-coordinate export, create and enable a Floor Area representing the main map boundary. The largest enabled valid Floor polygon is used as the reference source.

6. Reference Coordinates

Reference coordinates describe the same four physical map corners in the RMF and robot/world coordinate systems.

Index	Corner
0	Top-left
1	Top-right
2	Bottom-right
3	Bottom-left

6.1 Before Export

12. Import the map.
13. Verify resolution and origin.
14. Create and enable the main Floor Area.
15. Ensure at least four unique outer vertices.
16. Verify Building Name.
17. Confirm Auto Reference Coordinates is ready.
18. Export Reference Coordinates YAML or RMF Bundle.

6.2 Units

Current behavior

robot[] uses world/robot coordinates in meters. rmf[] is also exported in meters after reference-image conversion and resolution scaling.

```
reference = worldToReferenceImage(worldPoint)
rmf_x_m = reference_x_px * resolution
rmf_y_m = -reference_y_px * resolution
```

6.3 Output

```
reference_coordinates:
  automation_room1:
    rmf:
      - [x1_m, y1_m]
      - [x2_m, y2_m]
      - [x3_m, y3_m]
      - [x4_m, y4_m]
    robot:
      - [x1_m, y1_m]
      - [x2_m, y2_m]
      - [x3_m, y3_m]
      - [x4_m, y4_m]
```

7. RMF .building.yaml Export

The building exporter produces Traffic Editor style reference-image data while preserving lane direction, orientation, graph connectivity, object parameters, and building geometry.

```
coordinate_system: reference_image
levels:
  L1:
    drawing:
```

```

filename: <name>.png
floors: ...
lanes: ...
vertices: ...
walls: ...
lifts: {}
name: <name>

```

7.1 Lane Parameters

```

bidirectional: [4, true]
graph_idx: [2, 0]
orientation: [1, forward]
speed_limit: [3, 1.0]

```

7.2 RMF Lane Vertex Deduplication (Updated)

Connected lane endpoints can differ by tiny floating-point amounts after drawing or dragging. If exported independently, Traffic Editor may display two overlapping graph vertices at what should be one junction.

The latest exporter normalizes lane graph vertices in world meters. Lane endpoints within 0.02 m (2 cm) are treated as the same RMF graph vertex and reuse one vertex index.

```

Before export fix:
Lane A -> vertex 12
Lane B -> vertex 19 # visually same location

After export fix:
Lane A -> vertex 12
Lane B -> vertex 12 # one shared RMF junction

```

Important after updating

Export a NEW .building.yaml or RMF Bundle. Previously exported files still contain their old vertex list and are not modified automatically.

8. Save / Open .amrmap

Use .amrmap to preserve the complete editable project state.

8.1 Lane Orientation Persistence

Forward/Backward lane orientation is saved with each NavigationPath and restored when the .amrmap project is opened again.

```

{
  "id": "P-01",
  "type": "bidirectional",
  "orientation": "forward",
  "points": [...],
  "enabled": true
}

```

```
}
```

Compatibility

Older .amrmap files that do not contain orientation can still load; the orientation defaults to None.

9. GeoJSON and RMF Bundle

9.1 GeoJSON

GeoJSON is used for graph/coordinate exchange. One-way paths create one directed edge; bidirectional paths create forward and reverse edges.

9.2 RMF Bundle

```
<name>.building.yaml
<name>.png
<name>-navigation.json
<name>-reference-coordinates.yaml
README.txt
```

If a valid reference Floor is unavailable, the bundle may omit the reference-coordinate file while still exporting other RMF assets.

10. Export Formats

Output	Purpose
.amrmap	Reopen complete editable project
ROS Map ZIP	ROS YAML + occupancy image
Occupancy PNG	Preview / RMF drawing image
PGM	ROS occupancy map
YAML	ROS map metadata
Waypoints JSON	Navigation objects
Paths JSON	Navigation paths
Zones JSON	Zone definitions
Navigation JSON	Objects + paths + zones + robot config
GeoJSON	Graph / coordinate exchange
.building.yaml	Open-RMF / Traffic Editor
Reference Coordinates YAML	4 RMF <-> robot corner pairs in meters
RMF Bundle ZIP	RMF building + image + navigation + optional reference coordinates

11. Validation Before Deployment

- Objects are inside map bounds and outside obstacles.
- Path widths suit the robot footprint.
- Visual crossings have actual topology junctions when required.
- One-way / Bidirectional settings are correct.

- Forward / Backward lane orientation is correct.
- Pickup / Drop-off RMF names are correct.
- Floor geometry matches the intended environment.
- Reference coordinates contain exactly four paired corners.
- RMF reference coordinates are in meters.
- Exported RMF lane junctions do not contain unintended overlapping vertices.

12. Troubleshooting

12.1 Duplicate Lane Points in Traffic Editor

19. Confirm the latest buildingExporter.ts is in the project.
20. Rebuild the application.
21. Export a NEW .building.yaml or RMF Bundle.
22. Open the newly exported file in Traffic Editor.
23. Do not use an older .building.yaml when checking the fix.

```
rm -rf dist node_modules/.vite
npm run build
npm run dev
```

Why old files still show the problem

The fix changes how new RMF vertex lists are generated. It cannot rewrite .building.yaml files that were already exported.

12.2 Lane Orientation Missing After Reopening .amrmap

Check that the path has orientation set to "forward" or "backward" and that the current project schema includes the orientation field. Re-save the .amrmap after updating the application.

12.3 Reference Coordinates Not Generated

- Create a Floor Area.
- Enable it.
- Use at least four unique vertices.
- Verify the intended Floor is the largest enabled valid Floor.

12.4 Reference YAML Shows Nested Arrays

The current reference-coordinate exporter writes one compact coordinate pair per line:

```
- [x, y]
```

13. RMF Nav Graph Export

Version v0.11.5 adds direct RMF nav graph generation in the browser. The editor can export nav_graphs YAML files from the same NavigationPath and NavigationObject data used by the RMF building exporter, so a normal project does not need a separate ROS2 command just to create the graph YAML.

Traditional workflow

AMR Map Editor -> .building.yaml -> building_map_generator nav -> nav_graphs/0.yaml

Direct editor workflow

AMR Map Editor -> RMF Nav Graphs (.zip) -> nav_graphs/0.yaml, 1.yaml, 2.yaml ...

The direct exporter reproduces the graph data needed by the current AMR Map Editor workflow: graph vertices, directed lanes, speed limits, orientation constraints, navigation-object properties, shared-junction deduplication, and graph separation by Graph Index.

13.1 Nav Graph YAML Structure

Each generated graph file follows this structure:

```
building_name: automation_room1
levels:
  L1:
    lanes:
      - - 0
        - 1
        - {speed_limit: 1}
    vertices:
      - - 31.063379939265328
        - -41.54342948833109
        - {amr_object_type: waypoint, name: WP-01}
```

Bidirectional paths create two directed lanes. A path with a Forward or Backward orientation is exported with orientation_constraint. Intermediate path vertices use {name: ""} when they do not correspond to a named navigation object.

13.2 Exporting Nav Graphs

1. Create navigation objects and paths.
2. Connect path junctions and verify topology.
3. Set One-way/Bidirectional and Forward/Backward as required.
4. Assign Graph Index to each Path.
5. Open Export and select RMF Nav Graphs (.zip).
6. Extract the ZIP and use the files under nav_graphs/.

14. Multiple Nav Graphs and Graph Index

Each NavigationPath now has a Graph Index. The exporter groups enabled paths by this value and creates one YAML file per graph index.

14.1 Setting Graph Index

Select a Path, then use PATH PROPERTIES -> Graph Index. New and legacy paths default to Graph Index 0.

Path	Graph Index	Generated file
lane_1	0	nav_graphs/0.yaml
lane_2	0	nav_graphs/0.yaml
lane_3	1	nav_graphs/1.yaml
lane_4	2	nav_graphs/2.yaml

If the project uses graph indices 0 and 2 but no path uses index 1, the exporter creates only 0.yaml and 2.yaml. Empty graph files are not generated.

14.2 .building.yaml Relationship

The same Graph Index is written into each corresponding building lane as graph_idx. Example:

```
graph_idx: [2, 1]
```

This means the lane belongs to graph index 1. The direct nav graph exporter then places that path only into nav_graphs/1.yaml.

14.3 Shared Vertices and Graph Isolation

Path vertices are deduplicated inside each graph. Connected endpoints that refer to the same logical junction reuse one vertex index. Different Graph Index values are processed separately, so topology from graph 0 does not accidentally become part of graph 1 or graph 2.

14.4 Compatibility

Older .amrmap projects that do not contain graphIndex are treated as Graph Index 0 when loaded. Re-save the project after assigning additional graph indices if you want the new values preserved.

14.5 Verification Recommendation

Direct browser export is designed to match the nav graph behavior used by this editor, but it is not a complete reimplement of every possible feature of rmf_building_map_tools. For production deployment, compare representative generated graphs against output from the official building_map_generator when introducing features such as multiple levels, lifts, doors, mutex groups, or additional lane parameters.

15. Deployment Checklist

- ☐ Map resolution correct
- ☐ Origin X/Y and yaw correct
- ☐ Navigation objects positioned correctly
- ☐ Path junctions are truly connected
- ☐ Travel direction correct
- ☐ Lane orientation correct
- ☐ Pickup/Drop-off RMF properties correct
- ☐ Floor/Wall geometry correct
- ☐ Reference coordinates contain exactly 4 pairs
- ☐ RMF reference values are in meters
- ☐ Newly exported building.yaml has no unintended duplicate lane vertices
- ☐ building.yaml and PNG filenames match
- ☐ Graph Index is correct for every path when multiple nav graphs are used
- ☐ Expected nav_graphs/N.yaml files are present
- ☐ Bidirectional paths contain both directed lanes
- ☐ One-way paths contain only the allowed direction
- ☐ Forward/Backward constraints match the intended robot heading
- ☐ Named vertices contain expected charger, pickup, drop-off, and waypoint properties
- ☐ Connected junctions reuse one logical vertex inside each graph
- ☐ Validation completed
- ☐ Target robot/RMF workflow tested